Learnings: Building an LLM from Scratch using Rust

Learnings so far from the Building LLM from scratch using Rust

Tokenization, building an LLM from scratch using Rust.

1 Part 1:

3 things:

1. Converting Text to numbers using Byte Pair Encoding (BPE)
2. How this step shapes everything the model can and cannot do.
3. How different vocabulary sizes effect compression, speed and training behaviour.

Questions that popped up during the study:

1. Tokenizer? → Tool used to convert text to numbers
2. BPE? Algorithms used in the tokenization process
3. Feste? → The LLM name that we are building
4. Transformer? → Deep learning architecture
5. Shakesphere? → Real one? or any name for a tech/software? → The Real one! and this model is trained on Shakesphere complete works.

1.1 Learning about Transformer

Question: What exactly is a transformer and difference between transformer and a language model?

Transformer is a deep learning architecture used in the language model. It basically has encoding-decoding structure and works on the "Attention Mechanism"

1.2 What is attention mechanism?

This method finds the relation between multiple words and how are they related with each other in a sentence.

Difference between transformer and a conventional NN: This Transformer is found as a replacement for RNN/CNN. The conventional neural networks process data sequentially which takes a lot of time and storage space where as Transformer model can process large amount of data points parallelly like finding relevance, encoding, decoding etc.

1.3 More about the transformer model:

Transformer model holds a large amount of context. Transformer model does paralelly processes parameters like grammar, emotion etc. It has multi-head self attention for finding and understanding relationship between words. It also does positional encoding to maintain the order of words in a sentence.

Example: Ramu goes to temple in his village. In this sentence the word "his" refers to the person name "Ramu". This is identified by the model using the self attention mechanism.

1.4 Applications:

1.4.1 Training a customer support chatbot, medical report summaries etc

1.5 Benefits:

1. Long range context
2. Concurrency/Parallelism
3. Scalability

Resources: About Transformers

1.6 What are we building?

'Feste' → A GPT 2 Style Architecture model built in complete Rust as in general using Tensorflow/PyTorch for building transofrmer model abstracts away (eliminates or gives a theoretical level idea) math so that we can focus on the architecture. But in Rust it includes low-level memory management, performance, borrow checker on ownership and memory (control over all these things) safety.

2 Why Tokenization matter?

2.1 Considering the following example:

- `["he", "llo"]` → say "he" represents tokenID 530 and "llo" tokenID 840 respectively
- `[" he", "llo "]` → say " he" represents tokenID 460 and "llo " tokenID 670 respectively
- `["ol", "leh"]` → say "ol" represents tokenID 91 and "leh" represents tokenID 133 respectively

Though the memory contains the word "hello", it can't reverse the string because of different tokenIDs in each scenario resulting in a mismatch. This is one of the reasons a simple LLM stuggles to perform operations on a string like reversing it. At word level tokenization is not efficient.

3 Algorithm used for Tokenization by Feste is BPE

The goal is to convert Text to TokenIDs → process neural networks → convert those tokenIDs back to text and generate the output.

Instead of using tokenization at the word level we would go to character level where each character is a token.

Byte level → 8bit (binary digits) represent exactly 256 values.

English characters use 1 byte each while others use multiple bytes. For example emojis use 4bytes.

Each byte gets it's own tokenID in the file

Why 256?

3.1 BPE Assigning

Character	Byte	Token ID
A	65	530
B	66	870
C	67	754
D	68	214
E	69	125
F	70	859
G	71	381
H	72	350

Character	Byte	Token ID
I	73	328
J	74	242
K	75	854
L	76	204
M	77	792
N	78	858
O	79	658
P	80	189
Q	81	704
R	82	532
S	83	132
T	84	130
U	85	195
V	86	323
W	87	338
X	88	617
Y	89	716
Z	90	127

3.2 Iteratively merges the most frequent adjacent tokens to new tokens

Example: "th" is frequently repeated and "t" and "h" has individual tokens assigned initially. Since they are a frequently repeated pair they are merged together and gets asigned with a single new token say 344.

Common words end up with own tokens whereas rare words break down into familiar byte sequences that tokenizer seen in other context.

Smaller vocabulary → fragmentation building words from small pieces. Larger vocabulary → more to find meaning if trained well enough and identify conceptual relation with other words that are generated next

4 Implementation Details:

Starting with 256 entries mapping hex-encoded bytes to IDs (0-255)

During training → add entries for each merge

Merge rules stored in a vector for order significance

5 Example

Before encoding "the" → should apply merge1(M1) first for token256 then apply merge50 (M50)

M1 → "th" $\langle 74 \rangle \langle 68 \rangle$ token 256 . . . M50 → "the" ⇒ token 256 + $\langle 65 \rangle$ = token 300

Training process is completely deterministic like count pairs, find frequently occurring pairs, merge them, repeat. This ensures in no randomness.

6 Training Performance

Bottleneck: Counting adjacent pairs in the corpus

Instead of single threaded implementation, using Rayon(Rust library) for parallel counting where chunks (chunk1,chunk2...etc) are divided and counted in parallel.

Each thread checks its chunk boundary and count pair between last token and next chunk's first token such that no pairs are missed.

Sequential: Apply merge rules and updating corpus

Parallelism: Chunk boundary checks so that no pair is missed.

7 Parallelization using Rayon (Rust lib)

7.1 Parallel Processing Boundary Check Diagram

Below is a diagram illustrating how separate threads process distinct text chunks in parallel. The crucial step highlighted is the "Boundary Check," where Thread 1 looks at its last formed token and Thread 2's first formed token to identify pairs that might span across the chunk break (preventing missed merges like pre-existing "th" or "the" tokens).

thread 1 → `[["t","h"] ["e","a","d"]]` , thread 2 → `[["u","s"] ["e","d","t"]]` ... thread n `[][]`

Basically this is a parallel processing method for multi-thread operations where each thread checks the chunk boundary to see if it is missing any pairs to merge like "th" and "the" (from thread 1) and "us", "use", "used" (from thread 2)

This multi-thread operation helps in speeing up the process. Each thread checks the chunk boundaries so that no pair gets missed out. We only focus on the bottleneck which is counting pairs and not on speeding up the max performance like applying merges or updating the corpus as optimizing them complicates the code.

8 Encoding

Consider the following example:

Vocabulary: "To be"

Their hex values are as follows

Character	Hex Value
T	54
o	6f
Space	20
b	62
e	65

Now the list is : `[`⟨54⟩`,` ⟨6f⟩`,` ⟨20⟩`,` ⟨62⟩`,` ⟨65⟩`]`

Say the model learned 3 merges in the order:

M1: ⟨6f⟩⟨20⟩ → "o " → Token 256

M2: ⟨54⟩⟨6f⟩ → "To" → Token 257

M3: Token 256 + ⟨62⟩ → "o b" → 258

Encoding follows a sequential order while applying these merge rules:

From M1, the list will be updated from `[`⟨54⟩`,` ⟨6f⟩`,` ⟨20⟩`,` ⟨62⟩`,` ⟨65⟩`]` to `[`⟨54⟩`, 256,` ⟨62⟩`,` ⟨65⟩`]`

From M2, the list will remain same because in this case, it looks for applying the M2 to the updated list, but it can't find the hex value ⟨6f⟩ as it was updated to token 256 during the M1.

From M3, it looks for Token 256 and hex ⟨62⟩ and both exist in the newly updated list. So this rule will be applied to the list and now it becomes `[`⟨54⟩`, 258,` ⟨65⟩`]`

To summarize the encoding process, basically the model learns some merge rules and apply them sequentially to a list. If merge rules aligns with that list then the list is updated, if a merge rule could not find the token/hex value to apply the merge then the list won't get updated and will remain the same and it will skip to the next merge rule.

9 Decoding

Token	Map to	Hex Byte	Expected Output
⟨54⟩	→	⟨54⟩	T
258	→	⟨6f⟩⟨20⟩⟨62⟩	o b
⟨65⟩	→	⟨65⟩	e

After mapping the token IDs back to their corresponding hex byte values, join all those hex bytes together $\langle 54\rangle\langle 6f\rangle\langle 20\rangle\langle 62\rangle\langle 65\rangle$ and parse them back to the text ("To be") as per the UTF-8 standards.